Bài 4.
LÀM VIỆC VỚI FILE HỆ THỐNG (FS)

Nội dung

- Quản lý thư mục chứa file hệ thống
- Đọc files
- Ghi files
- Quản lý files upload

Giới thiệu

- Trong Node.js, làm việc với file là một nhu cầu rất phổ biến.
- Node cung cấp **module fs** thao tác với **file** và **thư mục** trên máy.
- Với fs, có thể:
  - đọc file
  - ghi file
  - tạo file/thư mục
  - xóa file/thư mục
- Nhờ đó, xử lý dữ liệu, lưu trữ thông tin hoặc xây dựng nhiều chức năng thực tế trong ứng dụng.

Vai trò của tập tin hệ thống

Hệ thống file thường được dùng để:

- Lưu trữ cấu hình ứng dụng
- Ghi và đọc log hệ thống
- Quản lý dữ liệu người dùng (ảnh, file upload, tài liệu)
- Xuất báo cáo (PDF, CSV, JSON, …)

fs module trong Node.js

- fs (File System) là module lõi (core module) của Node.js
- Cho phép tương tác trực tiếp với hệ thống tập tin của hệ điều hành
- Hỗ trợ:
  - Bất đồng bộ (asynchronous – callback): Không chặn chương trình, chạy xong sẽ gọi lại (callback)
  - Đồng bộ (synchronous – Sync): Chạy lần lượt, phải xong mới làm tiếp
  - Promise (fs/promises): dùng Promise – cách viết hiện đại, dễ đọc hơn, thường dùng với async/await.

Một số phương thức

Đọc – ghi nội dung file

STT	Phương thức	Chức năng
1	fs.readFile()	Đọc nội dung một file có sẵn
2	fs.writeFile()	Ghi đè nội dung file hoặc tạo file mới
3	fs.appendFile()	Ghi thêm nội dung vào cuối file
4	fs.open()	Mở file để đọc hoặc ghi với cờ (flag)

Một số phương thức

Thao tác trên file

STT	Phương thức	Chức năng
5	fs.rename()	Đổi tên hoặc di chuyển file
6	fs.copyFile()	Sao chép file
7	fs.link()	Tạo liên kết (hard link) tới file

Một số phương thức

Lấy thông tin & kiểm tra file

STT	Phương thức	Chức năng
8	fs.stat()	Lấy thông tin chi tiết của file
9	fs.exists()	Kiểm tra file tồn tại (giới thiệu, không khuyến khích dùng)
10	fs.access()	Kiểm tra quyền truy cập file

Một số phương thức

Tạo – xóa – đọc thư mục

STT	Phương thức	Chức năng
11	fs.mkdir()	Tạo thư mục mới
12	fs.rmdir()	Xóa thư mục
13	fs.readdir()	Đọc danh sách file trong thư mục

Một số phương thức

Quyền truy cập

STT	Phương thức	Chức năng
14	fs.chown()	Thay đổi quyền sở hữu file
15	fs.utimes()	Thay đổi thời gian truy cập/chỉnh sửa

Mở file trong Node.js

Hàm mở file: fs.open()

- **Cú pháp**: **fs.open(path, flags[, mode], callback)**
- **Ý nghĩa tham số:**
  - path: đường dẫn tới file
  - flags: chế độ mở file
  - mode: quyền truy cập
  - callback: hàm xử lý sau khi mở file
- Ví dụ: **fs.open('data.txt', 'r', callback)**

Mở file trong Node.js

Ví dụ

```
fs.open('data.txt', 'r', (err, fd) => {
        if (err) throw err; // thông báo lỗi – dừng
        console.log('Mở file thành công'); // in thông báo
});
```

Mở file trong Node.js

Ví dụ 1: Mở file chỉ để đọc (r)

```
const fs = require('fs’);
fs.open('data.txt', 'r', (err, fd) => {
        if (err) {
            console.error('Không mở được file:', err.message);
            return;
        }
        console.log('Mở file để đọc thành công, fd =', fd);
});
```

Mở file trong Node.js

Ví dụ 2: Mở file để ghi – ghi đè (w)

```
const fs = require('fs');
fs.open('output.txt', 'w', (err, fd) => {
        if (err) throw err;
        console.log('File đã được mở để ghi');
});
```

Mở file trong Node.js

Ví dụ 3: Mở file để ghi thêm (a)

```
const fs = require('fs');
fs.open('log.txt', 'a', (err, fd) => {
        if (err) throw err;
        console.log('Mở file để ghi thêm log');
});
```

Mở file trong Node.js

Ví dụ 4: Mở file để đọc và ghi (r+)

```
const fs = require('fs');
fs.open('data.txt', 'r+', (err, fd) => {
        if (err) throw err;
        console.log('Mở file để đọc và ghi');
});
```

Mở file trong Node.js

Ví dụ 5: Mở file an toàn – lỗi nếu file đã tồn tại (wx)

```
const fs = require('fs');
fs.open('newfile.txt', 'wx', (err, fd) => {
	if (err) {
		console.error('File đã tồn tại, không ghi đè');
		return;
	}
	console.log('Tạo file mới an toàn');
});
```

Mở file trong Node.js

Ví dụ 6: Kết hợp mở file và ghi nội dung

```
const fs = require('fs');
fs.open('note.txt', 'w', (err, fd) => {
     if (err) throw err;
        fs.write(fd, 'Hello Node.js', (err) => {
    if (err) throw err;
        console.log('Đã ghi dữ liệu vào file');
        fs.close(fd, () => {});
   });
});
```

Mở file trong Node.js

Các chế độ mở file trong Node.js (Flags)

Flag	Mô tả
r	Mở file chỉ đọc. Nếu file không tồn tại sẽ báo lỗi
r+	Mở file để đọc và ghi. Tương tự nếu file không tồn tại sẽ báo lỗi
rs	Mở file để đọc file một cách đồng bộ
rs+	Mở file để đọc và ghi một cách đồng bộ.
w	Mở file để ghi. File mới sẽ được tạo nếu không có sẵn và bị ghi đè nếu có sẵn
wx	Tương tự như w nhưng sẽ lỗi nếu file không tồn tại

Mở file trong Node.js

Các chế độ mở file trong Node.js (Flags)

Flag	Mô tả
wx	Tương tự như w nhưng sẽ lỗi nếu file không tồn tại
w+	Mở file để đọc và ghi. File mới sẽ được tạo nếu không có sẵn và bị ghi đè nếu có sẵn
wx+	Tương tự như w+ nhưng sẽ lỗi nếu file không tồn tại
a	Mở file để chèn thêm dữ liệu. Nếu file không tồn tại sẽ tạo mới file
ax	Tương tự a nhưng sẽ lỗi nếu file không tồn tại
a+	Mở file để đọc và chèn thêm dữ liệu. Nếu file không tồn tại sẽ tạo mới file
ax+	Tương tự a+ nhưng sẽ lỗi nếu file không tồn tại

Quản lý thư mục chứa file hệ thống

Trong ứng dụng server:

- Cần tạo thư mục lưu dữ liệu
- Kiểm tra thư mục tồn tại
- Đọc danh sách file
- Xóa, đổi tên thư mục

Quản lý thư mục chứa file hệ thống

Tạo thư mục

```
const fs = require('fs');
fs.mkdir('uploads', { recursive: true }, (err) => {//recursive tạo thư mục cha nếu chưa có
  if (err) throw err;
  console.log('Directory created');
});
```

Quản lý thư mục chứa file hệ thống

Kiểm tra thư mục tồn tại

```
const fs = require('fs');
fs.access('uploads', (err) => {
if (err) {
        console.log('Chưa tồn tại’);
        }
else {
        console.log('Đã tồn tại’);
    }
});
```

Đọc file

Cú pháp: fs.readFile(path[, options], callback)

Ý nghĩa:

- path: đường dẫn tới file
- options: encoding (ví dụ: 'utf8’)
- callback(err, data):
  - err: lỗi nếu có
  - data: nội dung file

Đọc file

Cú pháp: fs.readFile(path[, options], callback)

Ví dụ:

```
const fs = require('fs');
fs.readFile('data.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Lỗi đọc file:', err.message);
    return;
  }
  console.log('Nội dung file:', data);
});
```

⇒**Notes**:

- Trong vd không cần mở file để đọc mà NodeJs tự động mở
- Khi cần kiểm soát như: đọc/viết nhiều lần trên 1 file thì dùng **fs.open()**

Ghi file (ghi đè)

Cú pháp: fs.writeFile(path, data[, options], callback)

Ý nghĩa:

- path: đường dẫn tới file
- data là dữ liệu cần ghi vào file
- options: encoding (ví dụ: 'utf8’)
- callback(err, data):
  - err: lỗi nếu có
  - data: nội dung file

Đặc điểm:

- Nếu file chưa tồn tại → tạo mới
- Nếu file đã tồn tại → ghi đè toàn bộ nội dung
- Node.js mở file với flag w

Ghi file (ghi đè)

Cú pháp: fs.writeFile(path, data[, options], callback)

Ví dụ:

```
const fs = require('fs');
fs.writeFile('output.txt', 'Hello Node.js', (err) => {
  if (err) {
    console.error('Lỗi ghi file:', err.message);
    return;
  }
  console.log('Ghi file thành công');
});
```

Ghi file (Ghi thêm nội dung)

Cú pháp: fs.appendFile(path, data[, options], callback)

Đặc điểm:

- Ghi thêm vào cuối file
- Không làm mất dữ liệu cũ
- Node.js mở file với flag a

Ghi file (Ghi thêm nội dung)

Cú pháp: fs.appendFile(path, data[, options], callback)

Ví dụ:

```
const fs = require('fs');
fs.appendFile('log.txt', 'User login\n', (err) => {
  if (err) {
    console.error('Lỗi ghi log');
    return;
  }
  console.log('Ghi log thành công');
});
```

Xóa file

Cú pháp: fs.unlink(path, callback)

Ví dụ:

```
const fs = require('fs');

fs.unlink('./test.txt', (err) => {
if (err) {
console.error('Lỗi xóa file');
return;
}
console.log('Xóa file thành công');
});
```

Upload file

- Người dùng upload các loại file như: ảnh, tài liệu, file đính kèm
- Server cần:
  - Lưu file vào thư mục
  - Quản lý tên file
  - Tránh ghi đè

Upload file

- **Các bước thực hiện:**
  - Tạo thư mục lưu file
  - Đổi tên file để tránh trùng
  - Lưu thông tin file (tên, đường dẫn)

Upload file

Lưu file upload

```
const fs = require('fs');

const fileName = Date.now() + '_note.txt';
const filePath = 'uploads/' + fileName;
const fileBuffer = Buffer.from('Xin chào Node.js');

fs.writeFile(filePath, fileBuffer, (err) => {
if (err) {
console.error('Lỗi lưu file');
return;
}
console.log('Lưu file thành công');
});
```

The end